Pricing-driven Resource Allocation in the Computing Continuum Technical Report

Version 1.0

Alejandro García-Fernández[1], Boris Sedlak[2], José Antonio Parejo[1], Pantelis Frangoudis[3], Antonio Ruiz-Cortés[1], Schahram Dustdar[2 3]

[1] SCORE Lab, I3US Institute, Universidad de Sevilla, Sevilla, Spain

{agarcia29,japarejo,aruiz}@us.es

[2] Distributed Intelligence and Systems Engineering Lab, Universitat Pompeu Fabra, Barcelona, Spain

{boris.sedlak,schahram.dustdar}@upf.edu

[3] Distributed Systems Group, TU Wien, Vienna, Austria

{pantelis.frangoudis,dustdar}@dsg.tuwien.ac.at

ICSOC 2026

The 24th International Conference on Service-Oriented Computing

December 1-4, 2026
Łódź, Poland

July 12, 2026

INDEX

1 Introduction **2**

2 Background **3**
2.1 Configuration Problems in the Computing Continuum . . . 3
2.2 Automatic Analysis of iPricings . . . 4

3 Pricing-Driven Resource Allocation **5**
3.1 Problem Definition . . . 5
3.1.1 Offer . . . 5
3.1.2 Demand . . . 5
3.1.3 Binding Constraints . . . 5
3.2 Pricing-Driven Resource Allocation Workflow . . . 6
3.2.1 Offer Mapping . . . 6
3.2.2 Demand and Binding Constraints Encoding . . . 7
3.2.3 Optimization and Back-Projection . . . 7

4 Evaluation **9**
4.1 Experimental Setup . . . 9
4.1.1 **Dataset** . . . 9
4.1.2 **Synthetic Topology Generation** . . . 10
4.1.3 **Synthetic Demand Generation** . . . 10
4.2 Experimental Design . . . 12
4.2.1 Relation to research questions . . . 12
4.2.2 Application scenarios . . . 12
4.2.3 Deployment scales and scalability dimensions . . . 13
4.2.4 Provider configuration and additional constraints . . . 13
4.2.5 Baselines for RQ_4 . . . 14
4.3 Discussion of Results . . . 17
4.3.1 RQ_1. Modeling Expressiveness . . . 17
4.3.2 RQ_2. Optimization Capabilities . . . 17
4.3.3 RQ_3. Scalability . . . 18
4.3.4 RQ_4. Comparative Performance . . . 18
4.4 Threats to Validity . . . 26

5 Conclusions and Future Work **27**

Chapter 1

Introduction

Resource allocation (RA) in the computing continuum requires selecting infrastructure nodes –edge, fog, or cloud– that jointly satisfy a set of constraints (e.g. cost or capacity) [17]. As infrastructures grow in scale and heterogeneity, this decision space becomes inherently combinatorial [14]. Rather than an isolated challenge, this problem can be seen as an instance of the family of *configuration problems* (CPs), which aim to identify a configuration –i.e., a particular combination of candidates[1] and the bindings established among them– that satisfies a set of functional and non-functional requirements while respecting domain-specific constraints [7]. This perspective exposes two open gaps that motivate this work. First, current approaches rely on *ad-hoc* formalizations [18] that hinder reuse across other CPs. Second, they overlook constraints that are essential for resource allocation in multi-provider environments: interoperability agreements among providers, and coverage of required node types (e.g., cameras, sensors).

This work instantiates the CP framework for resource allocation by representing its configuration space as an *iPricing* –a model originally proposed in the SaaS domain [6]–. This provides the first evidence for the hypothesis in [7] that "a unified model could address CPs along the continuum", and advances the state of the art in resource allocation. The paper makes the following contributions:

1. A formulation of resource allocation in the computing continuum as an iPricing, instantiating the offer/demand/binding-constraints framework of [7].

2. PROMISE: A solution that leverages PRIME [5] –a pricing analysis tool– to explore the configuration space of RA problems and compute cost-optimal deployments in multi-provider, highly-constrained environments.

3. As a part of our evaluation, we define processes to synthetically generate realistic infrastructure offer and workload demands, enabling the construction of reproducible resource allocation scenarios.

4. A dataset of 9,600 precomputed scenarios covering different offers, demands, and optimization objectives, serving as a benchmark for future research [1].

The remainder of this report is organized as follows. Chapter 2 introduces the foundations of configuration problems and pricing models, and positions our work with respect to the state of the art. Chapter 3 presents the proposed pricing-based formulation for resource allocation together with PROMISE. Chapter 4 describes the experimental evaluation and discusses threats to validity. Finally, Chapter 5 concludes the paper and outlines future research directions.

[1] Hereinafter, *candidates* and *infrastructure nodes* will be used interchangeably.

Chapter 2

Background

2.1 Configuration Problems in the Computing Continuum

Recent work introduced the notion of Configuration Problems (CPs) as a unifying abstraction for a range of decision-making challenges across the computing continuum [7]. A CP consists of identifying a valid configuration, i.e., a combination of candidates and bindings between them, that satisfies a set of functional and non-functional requirements while respecting domain-specific constraints. The set of all valid configurations is referred to as the *configuration space*.

Under this perspective, CPs can be characterized through three dimensions: *offer* ($\mathcal{O}$), representing the available candidates and their attributes; *demand* ($\mathcal{D}$), representing the requirements that must be satisfied; and *binding constraints* (Γ), defining how candidates can be combined to form valid configurations [7].

Unfortunately, configuration spaces grow combinatorially as the number of candidates and requirements increases, making exhaustive exploration impractical. Consequently, CPs are rarely solved through brute-force exploration. Instead, most approaches progressively reduce the configuration space through a common workflow consisting of:

1. **Discovery.** Aims to identify the available candidates, their capabilities, and the constraints that govern their binding. The objective is to construct the initial configuration space from which valid configurations can later be derived. The larger the number of constraints, the more complex the construction of the configuration space becomes.

2. **Filtering.** The configuration space is reduced by discarding configurations that cannot satisfy the problem requirements and constraints [19]. Since the resulting configuration space may still remain excessively large, heuristic rules are often introduced to further restrict the search space by prioritizing promising alternatives. As a result, only a reduced subset of configurations is passed to subsequent stages.

3. **Optimization.** In some scenarios, selecting any feasible configuration may be considered an acceptable solution. However, when multiple alternatives exist, optimization techniques can be employed to identify those that best satisfy one or more quality objectives, such as latency, reliability, energy consumption, cost, etc. Since this process may be computationally expensive, particularly for large configuration spaces, it is often performed only after the preceding stages have reduced the search space to a manageable size.

4. **Monitoring.** Candidates, requirements, constraints and quality objectives may evolve over time, causing previously valid or optimal configurations to become unsuitable. In this stage solutions are proposed to track their evolution and trigger reconfiguration whenever necessary.

In this paper, we instantiate this framework for resource allocation. Specifically, infrastructure resources are treated as candidates, deployment requirements as demand, and infrastructure-specific restrictions as binding constraints. Furthermore, we focus on the filtering and optimization stages, using pricings as the representation formalism for the underlying configuration space.

2.2 Automatic Analysis of iPricings

A pricing is a component of a SaaS customer agreement [15] that constrains user access to *features* according to their subscription. A subscription typically consists of a *plan* and an optional set of *add-ons*, which determine the accessible features and may impose *usage limits* based on measurable *usage levels* [4]. The set of all valid subscriptions defines the software's *configuration space*, making the pricing a compact representation of it [6]. Figure 2.1 illustrates this structure.

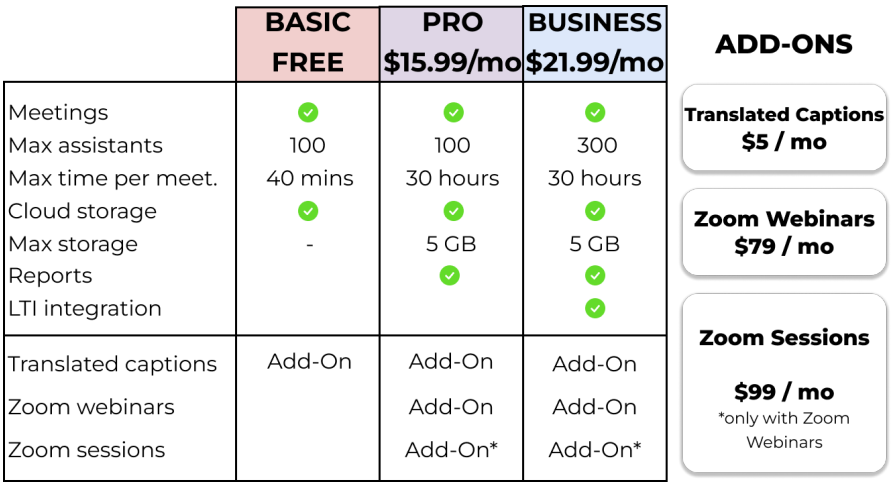


Figure 2.1: Sample pricing, containing: 10 features, 3 usage limits, 3 plans, 3 add-ons.

Recent work has extended pricings from static business documents to machine-oriented artifacts, termed *iPricings* [5], that enable –among other applications– automated analysis, validation, and optimization of configuration spaces. This makes iPricings particularly attractive for solving configuration problems, an therefore resource allocation.

Chapter 3

Pricing-Driven Resource Allocation

This section presents our pricing-driven approach to resource allocation. We first formulate resource allocation as a Configuration Problem. Then describe how this formulation is mapped onto an iPricing to enable the use of PRIME to identify cost-optimal deployment configurations.

3.1 Problem Definition

Following the CP formalization framework introduced in Sec. 2, we model the resource allocation problem in terms of offer, demand, and binding constraints.

3.1.1 Offer

represented as an infrastructure topology $\mathcal{O} = (\mathcal{N}, \mathcal{B})$, where $\mathcal{N}$ denotes the set of available nodes (candidates) and $\mathcal{B}$ the set of admissible bindings between them. Each node $n \in \mathcal{N}$ is characterized by (i) a resource vector $\mathbf{r}(n)$ describing the resources exposed by the node (e.g. memory, or storage); (ii) a context descriptor $\mathbf{c}(n) = \langle t(n), \ell(n), v(n), p(n) \rangle$, capturing its type, geographic location, provider, and rental price; and (iii) a set operational modes $\mathcal{M}$, representing different trade-offs between resource capacity (e.g. computing power) and rental price. We assume that all this information has been obtained through prior infrastructure discovery mechanisms, as described in [7].

3.1.2 Demand

Demand is represented as an aggregate resource vector $\mathcal{D}$ capturing the resources required to sustain the workload generated within a geographic region $\mathcal{Z}$. A deployment configuration $C \subseteq \mathcal{N}$ satisfies the demand iff

$$\sum_{n \in C} \mathbf{r}(n) \succeq \mathcal{D},$$

where $\succeq$ denotes component-wise dominance.

3.1.3 Binding Constraints

are the rules governing how nodes can be connected to form valid configurations. We represent them as $\Gamma = \langle N_{\max}, L_{\max}, P_{\max}, \mathcal{V} \rangle$, where $N_{\max}$ bounds the number of selected nodes, $L_{\max}$ their admissible distance from $\mathcal{Z}$ –which proxies latency–, $P_{\max}$ the deployment budget, $\mathcal{V}$ the set of allowed providers, T the set of required node types. Formally, C is valid iff

$$
\begin{aligned}
&\text{(i)}\ |C| \leq N_{\max}, \\
&\text{(ii)}\ \forall n \in C:\ L(n, \mathcal{Z}) \leq L_{\max}, \\
&\text{(iii)}\ \sum_{n \in C} p(n) \leq P_{\max}, \\
&\text{(iv)}\ \forall n \in C:\ v(n) \in \mathcal{V} \\
&\text{(v)}\ \forall \tau \in T,\ \exists n \in C : t(n) = \tau
\end{aligned}
$$

where $L(n, \mathcal{Z})$ denotes the maximum distance between a node n and any point of the polygon $\mathcal{Z}$. If $\ell(n)$ lies within $\mathcal{Z}$, the distance is defined as zero.

Definition (Resource Allocation Problem)

Given $\mathcal{O}$, $\mathcal{D}$, and Γ, the resource allocation problem consists of finding a valid configuration C^* that satisfies $\mathcal{D}$ while minimizing deployment cost.

3.2 Pricing-Driven Resource Allocation Workflow

To address the problem defined above, we propose a workflow that represents each topology configuration space as an iPricing and leverages PRIME (see Sec. 2) to analyze the resulting model in order to compute cost optimal deployment configurations that satisfy the specified *demand* and *binding constraints*.

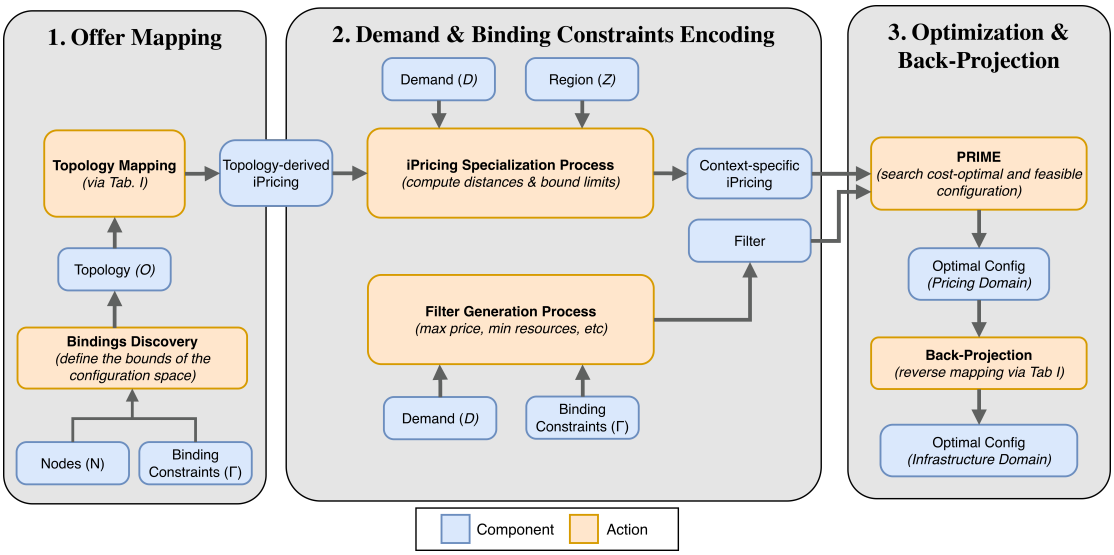


Figure 3.1: Pricing-driven resource allocation workflow

Figure 3.1 summarizes the proposed workflow in three stages: (1) offer mapping, which projects $\mathcal{O}$ onto the pricing domain; (2) problem encoding, where the generated iPricing is constrained based on the demand and binding constraints; and (3) optimization and back-projection, where PRIME identifies a globally optimal configuration that is then mapped back to the infrastructure domain.

3.2.1 Offer Mapping

In this stage, admissible bindings are first inferred from the available nodes and binding constraints to construct the topology $\mathcal{O}$. The resulting topology is then projected onto the pricing domain according to the mapping in Tab. 3.1, yielding an iPricing that compactly encodes its configuration space.

Table 3.1: Mapping between infrastructure and pricing domain

Infrastructure Domain	Pricing Domain
Currency	Currency
Node	Add-on
Node Types	Features
Distance	Usage Limits
Node Resources	Usage Limits
Provider Exclusions	Add-on Exclusions
Resources Unitary Price	Price Expression

However, the resulting iPricing cannot yet be directly processed by PRIME, as the monthly price of each add-on is not fully instantiated. Nodes in $\mathcal{N}$ typically expose more capacity than what is required by a concrete demand scenario; therefore, associating a fixed price to selecting a node would be misleading. Instead, unitary resource costs are considered to construct symbolic price expressions that define the cost of an add-on as a function of the effective resources it contributes. For example, a node offering memory at \$1.1/GB and storage at \$0.02/GB is associated with a price expression of the form:

$$node\_price = requested\_ram \cdot 1.1 + requested\_storage \cdot 0.02 \tag{3.1}$$

These expressions are left unresolved and will be instantiated in the next stage once demand information becomes available.

3.2.2 Demand and Binding Constraints Encoding

In this stage, the demand vector $\mathcal{D}$, the region $\mathcal{Z}$, and the binding constraints Γ are jointly used to (i) specialize the topology-derived iPricing to the concrete deployment context, and (ii) derive a filter that constrains the configuration space explored by PRIME.

Regarding (i), the transformation is driven by $\mathcal{D}$ and $\mathcal{Z}$. For each node, its distance to $\mathcal{Z}$, denoted by $L(n, \mathcal{Z})$, is encoded as a usage limit of the corresponding add-on. In addition, usage limits representing resources are capped according to the corresponding demand. Formally, for each add-on a and resource i, the effective usage limit is defined as: $u_i^a = \min\left(u_{i,0}^a, \mathcal{D}_i\right)$, ensuring that no add-on contributes capacity beyond what is required by $\mathcal{D}$. Based on these bounded usage limits, the price expressions of each add-on are then evaluated, yielding a concrete monthly price that reflects the effective resources contributed by selecting that node. For instance, if we consider the following demand vector:

$$\mathcal{D} = \langle \mathcal{D}_{ram}, \mathcal{D}_{sto} \rangle = \langle 20, 100 \rangle$$

the price expression from Eq. 3.1 would be instantiated as follows:

$$20 \cdot 1.1 + 100 \cdot 0.02 = \$24$$

Regarding (ii), the filter is obtained by combining $\mathcal{D}$ and Γ, which respectively induce minimum usage limits and bound the constraints defined in Sec. 3.1.

3.2.3 Optimization and Back-Projection

Artifacts generated in (2) are provided to PRIME [5], which first uses them to delimit the admissible configuration space, and then searches an optimal configuration that satisfies all requirements while minimizing the total deployment cost. Finally, the resulting solution is

mapped back to the infrastructure domain by applying Tab. 3.1 in reverse, yielding the optimal set of nodes on which to deploy the workload/services.

Chapter 4

Evaluation

The goal of our evaluation is to assess both the modeling capabilities and the computational viability of our proposal. To this end, we defined the following research questions (RQs):

- ***RQ*$_1$ *(Modeling Expressiveness)**: Is the proposed pricing-based formulation expressive enough to model realistic multi-provider scenarios involving interoperability constraints and heterogeneous infrastructure?*
- ***RQ*$_2$ *(Optimization Capabilities)**: Can the approach identify cost-optimal deployments that satisfy a set of given constraints within a reasonable time?*
- ***RQ*$_3$ *(Scalability)**: How does the approach scale with increasing numbers of clients and candidate nodes?*
- ***RQ*$_4$ *(Comparative Performance)**: How does the proposed approach compare against state-of-the-art resource allocation techniques in terms of performance, number of selected nodes, and constraint-modeling capacity?*

4.1 Experimental Setup

4.1.1 Dataset

Our evaluation is based on the Edge User Allocation (EUA) dataset [10], which provides the geographic distribution of 95,562 edge servers (hereinafter, nodes) across Australia –primarily in the Melbourne metropolitan area– together with the location of 4,748 clients. Since our goal is to generate realistic topologies rather than to model user behavior directly, we rely exclusively on the infrastructure-related information in the dataset. The rationale behind discarding client-related data is discussed in more detail in Sec. 4.1.3.

Given this objective, most of the original node attributes are discarded, retaining only: *latitude*, *longitude*, *elevation*, and the node *name*. The resulting dataset is then preprocessed to incorporate some information required for the evaluation:

- *Provider inference.* The *name* attribute is used to identify nodes containing information about their provider, restricted to: Optus, Telstra, Vodafone, Macquarie, and Telecom. Out of the 95,562 nodes in the dataset, 18,822 contain such information, which we consider sufficient for evaluation purposes. Once the inference is completed, the original *name* attribute is discarded and replaced by an explicit *provider* field.
- *Synthetic resource generation.* As the EUA dataset does not include hardware specifications, each node is further enriched with a synthetic resource capacity vector ($\mathbf{r}(n)$). To this

end, nodes are randomly assigned to one of three infrastructure tiers –edge, fog, or cloud– reflecting increasing levels of computational capacity. For each tier, resource capacities are generated pseudo-randomly using a predefined configuration to ensure reproducibility.[1] We consider the following resource dimensions: RAM, storage, CPU, GPU, and TPU. Resource prices are assigned per unit and may vary across providers and infrastructure tiers.

4.1.2 Synthetic Topology Generation

Synthetic topologies are generated by spatially sampling the enriched EUA dataset (see Sec. 4.1.1). Each topology is defined by specifying the geographic coordinates (latitude, longitude, and elevation) of a center point within the Melbourne area, together with a radius that determines a three-dimensional spherical region. All nodes whose location falls within this region are selected as candidates for the topology.

In addition to the spatial parameters, a maximum number of nodes may be provided to cap the size of the resulting topology by randomly selecting a subset of nodes. In parallel, a set of business rules is specified to capture interoperability constraints and exclusion agreements among providers.

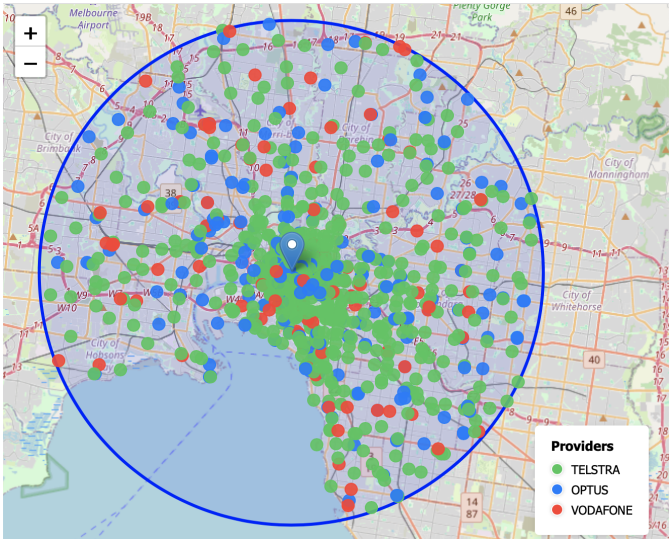


Figure 4.1: Sample topology generated around the Melbourne city center. Each point represents a node.

Finally, the selected nodes and the associated business rules are used as input to the "topology mapping" stage of our approach (see Sec 3.2), yielding an iPricing that represents the generated topology in the pricing domain. Figure 4.1 illustrates a sample topology produced using this process.

4.1.3 Synthetic Demand Generation

As discussed in Sec 4.1.1, we do not use the client-related information provided by the EUA dataset to derive demand. The dataset only reports the geographic location of clients, without any indication of the workload they generate or the quality-of-service requirements they impose. Moreover, even if we consider all clients in the dataset (4,748), this number is insufficient to represent large-scale deployments.

Instead, demand is generated synthetically in accordance with our problem formulation, which represents it as an aggregate resource demand vector $\mathcal{D}$ associated with a region of interest $\mathcal{Z}$. To this end, we define a function f_{res} that maps the number of users $N = |U|$ assumed to be

[1] Due to space constraints, further details of the generation procedure are omitted and made available in the accompanying artifact [1].

located within $\mathcal{Z}$ to a demand vector:

$$\mathcal{D} = f_{\text{res}}(N) = \langle \mathcal{D}_{\text{ram}}, \mathcal{D}_{\text{sto}}, \mathcal{D}_{\text{cpu}}, \mathcal{D}_{\text{gpu}}, \mathcal{D}_{\text{tpu}} \rangle$$

The objective of f_{res} is not to predict the exact requirements of a production deployment, but to generate *plausible, internally consistent, and reproducible* demand instances for evaluating our proposal across different scenarios.

- *User Activity and Throughput Characterization:* Not all users are active simultaneously. Let N denote the total number of users and N_a the number of active users. We model N_a as a binomial random variable:

  $$N_a \sim \text{Binomial}(N, p_s)$$

  where $p_s \in [0, 1]$ denotes the service concurrency level, a standard abstraction in capacity planning [13].

  Each active user generates requests at a certain rate. The per-user request rate is modeled as a random variable with a configurable mean, allowing moderate variability across users. The aggregate request arrival rate λ (requests per second) is then computed as:

  $$\lambda = \alpha_s \sum_{i=1}^{N_a} r_i$$

  where r_i denotes the request rate of user i and $\alpha_s \geq 1$ is a service-specific safety factor accounting for burstiness and modeling uncertainty, following established practices in large-scale system provisioning [2].

- *Memory Demand:* is decomposed into three service-specific components: a fixed baseline footprint, a shared component amortized across users, and a per-session overhead:

  $$\mathcal{D}_{\text{ram}} = \alpha_s \left(\mathcal{D}^{\text{base}}_{\text{ram},s} + \mathcal{D}^{\text{shared}}_{\text{ram},s}(N_a) + \mathcal{D}^{\text{session}}_{\text{ram},s} \cdot N_a \right)$$

  Memory usage is not assumed to scale linearly with the number of users. Empirical studies show that large portions of memory consumption –such as caches, shared models, or runtime state– are amortized across sessions [16]. Accordingly, the shared component is modeled as:

  $$\mathcal{D}^{\text{shared}}_{\text{ram},s}(N_a) = \beta_s \cdot \sqrt{N_a}$$

  where β_s is a service-specific scaling coefficient.

- *Compute and Accelerator Demand:* Compute resources are dimensioned based on throughput rather than per-user provisioning; accordingly, CPU demand is modeled as follows:

  $$\mathcal{D}_{\text{cpu}} = \frac{\lambda \cdot t^{\text{cpu}}_s}{u^{\text{cpu}}_s}$$

  where t^{cpu}_s is the average CPU service time per request and $u^{\text{cpu}}_s \in (0, 1)$ is the target utilization threshold.

  Accelerator demand is computed analogously. Let ϕ^{gpu}_s denote the fraction of requests executed on the GPU:

  $$\mathcal{D}_{\text{gpu}} = \phi^{\text{gpu}}_s \cdot \frac{\lambda \cdot t^{\text{gpu}}_s}{u^{\text{gpu}}_s}$$

  TPU demand is computed using the same formulation, with service-specific parameters adjusted to TPU execution. In all cases, accelerator demand is expressed in *equivalent processing units*, capturing sustained throughput rather than physical devices, and reflecting batching and multiplexing effects commonly observed in modern inference-serving systems [3].

- *Storage Demand:* as with memory demand, it is decomposed into three components: a fixed baseline, a per-session overhead, and an optional log retention term:

$$\mathcal{D}_{\text{sto}} = \alpha_s \left(\mathcal{D}^{\text{base}}_{\text{sto},s} + \mathcal{D}^{\text{session}}_{\text{sto},s} \cdot N_a + \lambda \cdot L_s \cdot W_s \right)$$

  In this formulation, L_s denotes the service-specific log volume generated per request and W_s the retention window.

Once demand has been generated independently for each resource dimension, the resulting values are combined to construct the aggregate demand vector $\mathcal{D}$. Further details on the specific values assigned to parameters in our evaluation are provided in the accompanying artifact [1].

4.2 Experimental Design

Our evaluation is based on a single, systematic experiment that uses the topology and demand generation strategies introduced in Sec. 4.1 to construct a set of realistic deployment scenarios that vary along three orthogonal dimensions: the type of the application to be deployed, the size of the offered topology (in terms of candidate nodes), and the intensity of the demand (in terms of the number of clients). Their combination yields deployment scenarios of increasing complexity on which the proposed pricing-driven workflow is evaluated. In total, the experiment comprises 9,600 distinct test cases.

4.2.1 Relation to research questions

RQ1 is addressed by assessing whether all deployment instances defined in the experiment can be represented using the proposed pricing-based formulation. RQ2 is evaluated by analyzing the proportion of test cases for which the approach successfully identifies a feasible and cost-optimal configuration. RQ3 is addressed by measuring the execution time required to solve each instance and examining how it evolves as the number of clients or candidate infrastructure nodes increases. Finally, RQ4 is addressed by comparing PROMISE against the 16 baselines introduced in Sec. 4.2.5 along four dimensions: feasibility under progressively stricter constraint sets, execution time, number of selected nodes, and solution cost.

4.2.2 Application scenarios

We consider four application scenarios, each deployed over a different area of the Melbourne region and characterized by distinct requirements:

- **A CCTV surveillance system** deployed around the Royal Melbourne Hospital. This scenario is characterized by a sustained high data throughput due to continuous video streams, combined with moderate latency requirements.

- **Virtual Reality (VR) service** deployed in the Melbourne city center to support interactive touristic experiences. This scenario requires ultra-low latency and high data rates to ensure an acceptable quality of experience.

- **Robotic arm coordination system** deployed in the Port of Melbourne to coordinate multiple robotic arms used for logistics and cargo handling. This scenario features strict latency constraints but comparatively lower data volumes.

- **LiDAR-based sensing and data collection system** deployed in the Sunbury area to support agricultural monitoring tasks. This scenario is dominated by very large data volumes, with less stringent latency requirements.

This setting follows the multi-scenario deployment perspective adopted by [8], which evaluated placement strategies under heterogeneous demand and service requirements.

4.2.3 Deployment scales and scalability dimensions

Beyond the application dimension, scalability in our experiment is explored by independently varying the size of the offered topology and the intensity of the demand. To this end, we define three scales: *small* (S), *medium* (M), and *large* (L).

For each application and deployment scale, we instantiate four increasing demand levels (number of clients) and four increasing bounds on the number of candidate nodes. Their values are derived from the ranges reported in Tab. 4.1, yielding a total of 96 distinct scenario types. For each scenario type, we then generate 100 topologies (see Sec. 4.1.2) together with corresponding demand workloads that satisfy the imposed constraints. This repeated sampling strategy enables a robust estimation of execution time by averaging results across structurally different yet semantically equivalent problem instances.

Table 4.1: Experimental scenarios considered in the evaluation. For each application and deployment scale, ranges indicate the minimum and maximum values used to generate four uniformly spaced configurations for the number of clients and candidate nodes. Node ranges depend only on the deployment scale, while user ranges are application-specific.

Scale	Application	Max. Users	Max. Nodes
S	CCTV	20–80	5–30
	VR	25–100	
	Robot	15–60	
	LiDAR	50–200	
M	CCTV	100–400	50–200
	VR	200–800	
	Robot	75–300	
	LiDAR	500–2000	
L	CCTV	500–2000	300–500
	VR	1500–8000	
	Robot	250–1000	
	LiDAR	2500–5000	

To isolate the impact of each scalability dimension when addressing RQ3, the two variables are varied independently. When analyzing scalability with respect to demand, the maximum number of candidate nodes is kept fixed to 20. Conversely, when analyzing scalability with respect to topology size, the number of clients is fixed to 100.

Table 4.2: Binding constraints applied per application scenario and deployment scale. CAM = camera, NET = network node, DC = data center, SEN = sensor, CPU = computer.

Scale	App.	Max. Distance (m)	Max. Nodes	Allowed Node Types
S	VR	3,750	4	CAM, SEN, NET, DC
	Robot	500	3	SEN, CPU, NET
	LiDAR	2,500	3	SEN, NET, DC
	CCTV	500,000	3	CAM, NET, DC
M	VR	7,500	6	CAM, SEN, NET, DC
	Robot	1,000	6	SEN, CPU, NET
	LiDAR	5,000	6	SEN, NET, DC
	CCTV	800,000	6	CAM, NET, DC
L	VR	15,000	10	CAM, SEN, NET, DC
	Robot	1,500	10	SEN, CPU, NET
	LiDAR	10,000	10	SEN, NET, DC
	CCTV	1,000,000	10	CAM, NET, DC

4.2.4 Provider configuration and additional constraints

All experiments consider three infrastructure providers: TELSTRA, OPTUS, and VODAFONE. Interoperability constraints are enforced such that VODAFONE nodes can interoperate with

both TELSTRA and OPTUS, while TELSTRA and OPTUS are mutually exclusive. It is important to note that they are fictitious and defined solely to create representative evaluation scenarios for the case study.

In addition, each test case is further restricted by a request that captures deployment-specific requirements. These include: (i) a maximum admissible distance between selected nodes and $\mathcal{Z}$, used as a proxy for latency constraints; (ii) an upper bound on the number of nodes that may compose the final deployment configuration; (iii) a maximum budget; and (iv) constraints on the types of nodes that may be selected, reflecting application-specific functional requirements.

These request parameters vary across application scenarios and deployment scales and are summarized in Tab. 4.2.

4.2.5 Baselines for RQ_4

PROMISE is compared against 16 state-of-the-art baselines drawn from three recent contributions to the edge–cloud–end resource allocation literature. **RAOM4CC** [14] contributes nine heuristics organized in two families: three single-layer variants that restrict placement to a single infrastructure tier, and six advanced variants that operate across tiers using different selection criteria. **EdgeWiseCR** [12] contributes six placement strategies –three declarative Prolog-based heuristics that select a single node, and three hybrid approaches that combine declarative reasoning with mixed-integer linear programming (MILP) to distribute demand across multiple nodes. Finally, **MS-GD-P** [9] addressed the problem using a priority-enhanced genetic algorithm that searches for solutions covering the largest possible number of users. Together with PROMISE, these form the 17 techniques evaluated throughout this section.

Since RAOM4CC and MS-GD-P do not provide public implementations, both were re-implemented from their published specifications. For EdgeWiseCR, we reused the authors' implementation, replacing its Prolog-based configuration filter with an equivalent Python implementation. In all cases, only lightweight input adapters were developed to translate our synthetic topologies into the format expected by each baseline, without modifying their optimization models.

RAOM4CC: Resource Allocation Optimization Model for Computing Continuum

RAOM4CC [14] formulates the deployment problem as a resource allocation optimization model for computing continuum scenarios. Its objective is to minimize a linear combination of financial cost, execution time, resource usage, and energy consumption. The approach operates in two phases: (i) resource allocation via heuristic selection, and (ii) task execution with load balancing. Because the original formulation does not enforce provider-level interoperability or device-type coverage constraints, its solutions must be re-evaluated against PROMISE's full constraint set when comparing feasibility.

The nine RAOM4CC variants differ in the tier to which they restrict placement and in the ordering criterion used for node selection:

- `one_layer_mist` – restricts placement to mist-tier (lowest-capacity) devices. Node selection follows a single-layer heuristic that assigns resources from the closest mist node satisfying demand. This variant is the cheapest but frequently lacks the GPU and storage capacity required by compute-intensive workloads.

- `one_layer_edge` – restricts placement to edge-tier devices. The selection heuristic mirrors `one_layer_mist` but operates on the middle tier, which offers a broader catalogue of device types (including cameras and sensors) and moderate computational capacity.

- `one_layer_cloud` – restricts placement to cloud-tier (highest-capacity) data centres. While this variant benefits from high GPU and storage availability, it cannot satisfy application scenarios that require edge-resident device types such as sensors or actuators.

- `round_robin` – a multi-layer variant that distributes demand across providers in round-

robin order, selecting one node per provider. This strategy does not consider resource adequacy and may co-locate mutually exclusive providers.

- `best_fit` – a multi-layer variant that selects, for each resource dimension, the node with the tightest capacity match (best-fit bin packing). This criterion minimizes wasted capacity but does not account for provider exclusion or device-type constraints.

- `best_fit_delay` – extends `best_fit` by incorporating a latency-aware ordering criterion: among nodes with adequate capacity, those with lower estimated response time are preferred.

- `best_fit_delay_energy` – further extends `best_fit_delay` by adding an energy-efficiency dimension to the selection criterion, preferring nodes that combine low latency with low energy consumption.

- `delay_heuristics` – selects nodes based solely on estimated response time, placing demand on the lowest-latency infrastructure regardless of capacity or provider compatibility.

- `delay_energy_heuristics` – combines delay and energy criteria in a weighted heuristic, selecting nodes that minimize a linear combination of response time and energy footprint.

The three single-layer variants (`one_layer_mist`, `one_layer_edge`, `one_layer_cloud`) select exactly one node by construction, which immunizes them against provider-exclusion violations but limits their ability to satisfy heterogeneous device-type requirements. The six advanced variants (`round_robin`, `best_fit`, `best_fit_delay`, `best_fit_delay_energy`, `delay_heuristics`, `delay_energy_heuristics`) may select multiple nodes but do not enforce provider exclusion or device-type coverage constraints.

EdgeWiseCR: Edge-Aware Configuration and Resource Allocation

EdgeWiseCR [12] tackles the edge service deployment problem through a two-stage architecture. A declarative Prolog-based engine first generates a candidate set of feasible node configurations by filtering the infrastructure against application requirements. A second-stage optimizer then selects the configuration that minimises deployment cost. The six EdgeWiseCR variants differ in the optimization strategy applied in the second stage:

- `prolog` – uses the Prolog-based configuration filter alone, selecting the first feasible single-node configuration without a subsequent optimization step. This variant is the fastest but does not minimize cost.

- `prolog_cr` – extends `prolog` by adding a cost-reliability ordering criterion: among feasible configurations, the one with the best cost-reliability trade-off is selected.

- `prolog_num` – extends `prolog` by preferring configurations with the fewest number of distinct resource types, reducing provisioning complexity.

- `edgewise` – replaces the Prolog-only selection with a MILP formulation that distributes demand across multiple nodes to minimize total deployment cost. This variant can select up to 10 nodes but does not enforce provider exclusion.

- `edgewise_cr` – extends `edgewise` by adding a reliability constraint to the MILP formulation, requiring that the selected configuration meets a minimum reliability threshold.

- `edgewise_num` – extends `edgewise` by incorporating a cardinality constraint that limits the number of selected nodes, trading cost optimality for reduced management overhead.

The three Prolog-based variants (`prolog`, `prolog_cr`, `prolog_num`) select a single node and are therefore immunized against provider-exclusion violations by construction. The three MILP-

based variants (`edgewise`, `edgewise_cr`, `edgewise_num`) distribute demand across multiple nodes (median 6, range 1–10), which enables them to handle heterogeneous resource requirements but exposes them to provider-exclusion violations.

MS-GD-P: Multi-Scenario Genetic Deployment with Priorities

MS-GD-P [9] addresses the edge–end service deployment problem through a priority-enhanced genetic algorithm. The approach encodes candidate deployments as chromosomes and evolves a population over multiple generations. A priority mechanism biases the search toward solutions that maximize the number of covered users, while a fitness function evaluates each candidate based on coverage, resource usage, and energy consumption. MS-GD-P operates on a heterogeneous infrastructure comprising cloud, edge, and end devices, and is the only baseline that jointly optimizes across all three tiers without restricting placement to a single layer.

In our evaluation, MS-GD-P selects the largest number of nodes among all baselines (median 16, range 3–80), reflecting its coverage-maximization objective. However, the genetic search does not enforce provider exclusion or device-type coverage constraints, which results in a steep feasibility drop when these constraints are activated. Its execution time is higher than the heuristic baselines but remains within the order of milliseconds for the scenario sizes considered.

Baseline Categorization

Although the 16 baselines originate from three independent contributions, their behavior in our evaluation reveals natural groupings based on structural similarities in how they select nodes and handle constraints. We organize them into five categories that reflect these behavioral patterns:

- **Single-node greedy heuristics** (8 techniques): the three EdgeWiseCR Prolog-based variants (`prolog`, `prolog_cr`, `prolog_num`) and five RAOM4CC multi-layer heuristics that select a single node by construction (`best_fit`, `best_fit_delay`, `best_fit_delay_energy`, `delay_heuristics`, `delay_energy_heuristics`).[2] These variants select exactly one node using a greedy, first-fit, or declarative filtering criterion. By construction, they cannot violate provider-exclusion constraints, but they are unable to satisfy heterogeneous device-type requirements that span multiple infrastructure tiers. They exhibit the fastest execution times (median $< 10^{-3}$ s) and the lowest solution costs, because they solve a structurally simpler problem.

- **Single-tier restricted placement** (3 techniques): the three **RAOM4CC** `one_layer_*` variants (`one_layer_mist`, `one_layer_edge`, `one_layer_cloud`). These restrict placement to a single infrastructure tier (mist, edge, or cloud, respectively) and select one node within that tier. Their feasibility depends critically on whether the target tier contains the device types required by the application: `one_layer_mist` fails for compute-intensive workloads because mist-tier devices lack GPU and storage capacity; `one_layer_cloud` fails for edge-resident device types (sensors, actuators) that do not exist in cloud data centres; and `one_layer_edge` achieves the broadest feasibility because edge nodes offer the most diverse device catalogue at moderate capacity.

- **Multi-node MILP-based placement** (3 techniques): the three **EdgeWiseCR** hybrid variants (`edgewise`, `edgewise_cr`, `edgewise_num`). These combine declarative configuration filtering with mixed-integer linear programming to distribute demand across multiple nodes (median 6, range 1–10). They can satisfy heterogeneous resource requirements but do not enforce provider exclusion, which causes their feasibility to collapse to 26–41% under PROMISE's full constraint set. Their execution times are higher than the greedy heuristics but remain below 10^{-2} s.

- **Multi-node heuristic placement** (1 technique): **RAOM4CC** `round_robin`. This variant distributes demand across providers in round-robin order, selecting one node per

[2] For readability, the five RAOM4CC variants in this category are shown in Tab. 4.4 as two aggregated rows: the three `best_fit_*` variants and the two `delay_*` variants share near-identical behavior.

provider without considering resource adequacy or provider compatibility. It may co-locate mutually exclusive providers and does not enforce device-type coverage constraints.

- **Priority-driven genetic search** (1 technique): **MS-GD-P**. This approach uses a genetic algorithm to maximize coverage across a heterogeneous infrastructure. It selects the most nodes (median 16, range 3–80) and exhibits the highest execution time among baselines (median 10^{-2}–10^{-1} s), but its feasibility under provider exclusion drops to 11.8% because the search does not check provider compatibility.

This categorization is reflected throughout the results discussion: PROMISE is compared both at the aggregated category level (Tab. 4.3) and at the individual variant level (Tab. 4.4–4.6), enabling a layered analysis of how constraint coverage, node count, execution time, and cost trade off across structurally different approaches.

4.3 Discussion of Results

All experiments were executed on a Mac Mini machine equipped with an Apple M4 Pro processor and 24 GB of RAM. The following discussion analyzes the obtained results in relation to the defined research questions.

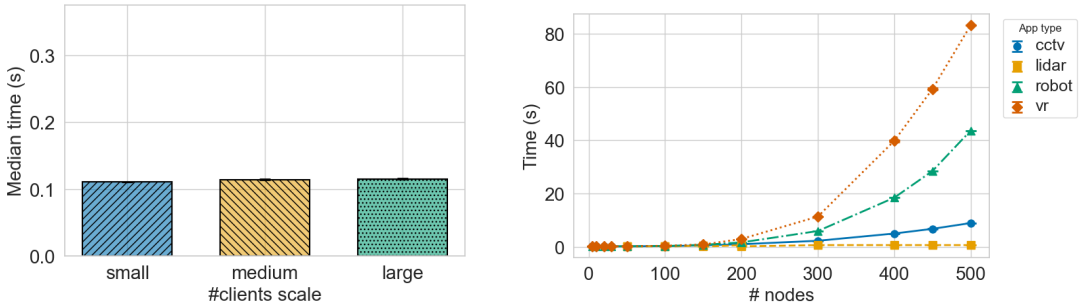
4.3.1 RQ_1. Modeling Expressiveness

The results provide strong evidence that the proposed pricing-based formulation is sufficiently expressive to model realistic multi-provider resource allocation scenarios. All 9,600 problem instances presented in Sec. 4.2 were successfully represented as iPricings without requiring ad-hoc adaptations. This was achieved through a programmatic generation of pricing configurations using the python reference implementation available at [1].

4.3.2 RQ_2. Optimization Capabilities

The reference implementation was able to compute a feasible and cost-optimal deployment for *all* test cases considered in the experiment.

In addition, we explicitly evaluated infeasible configurations by constructing request profiles whose constraints could not be jointly satisfied by the available infrastructure. In such cases, the solver consistently reported the absence of a solution, indicating that the approach correctly distinguishes between feasible and infeasible instances rather than failing silently or returning suboptimal configurations.



(a) Median execution time per #clients scale

(b) Median execution time vs increasing number of candidate nodes

Figure 4.2: Execution time of the proposed approach under increasing problem size. Each point represents the median solving time computed over 100 independent topology instances generated for the corresponding experimental configuration. Error bars denote 95% confidence intervals around the median. Due to the low intra-scenario variability observed in the experiments, these intervals are often visually negligible. A detailed statistical analysis is provided in [1].

Execution time of the proposed approach under increasing problem size. Each point represents the median solving time computed over 100 independent topology instances generated for the corresponding experimental configuration. Error bars denote 95% confidence intervals around the median. Due to the low intra-scenario variability observed in the experiments, these intervals are often visually negligible. A detailed statistical analysis of runtime stability and distributional properties is provided in the accompanying lab package.

4.3.3 RQ_3. Scalability

Figure 4.2 illustrates the evolution of execution time as the two scalability dimensions considered –i.e the number of clients and candidate nodes– increase. Each value correspond to the median execution time obtained across the 100 topology instances generated for each scenario.

On the one hand, Figure 4.2a demonstrates that execution time remains stable as the number of clients grows. This behavior is expected, since scaling demand only tightens constraints (e.g., higher usage limits), but does not introduce additional decision variables nor expand the search space. In short, demand impact on execution time is negligible.

On the other hand, Figure 4.2b shows that execution time increases with the number of candidate nodes, as expected due to the expansion of the decision space. However, this growth is scenario-dependent. Applications with more specialized resource requirements –such as VR, which relies on relatively scarce resources like GPUs– exhibit steeper increases than scenarios dominated by widely available resources (e.g., storage). This indicates that solver performance is strongly influenced by the availability and distribution of the required resources.

Despite this trend, execution times remain within practical bounds. Problem instances involving up to 200 candidate nodes are consistently solved in *under three seconds*, making the approach suitable for real-world planning and decision-support settings. Even for larger infrastructures, execution times stay in the order of seconds rather than minutes.

Overall, these results show that the proposed approach is expressive enough to model all the considered multi-provider scenarios (RQ_1), consistently identifies feasible and cost-optimal solutions when they exist (RQ_2), scales favorably to realistic infrastructure sizes, handling a broad range of deployment scenarios within practical time limits (RQ_3), and dominates the baselines on feasibility while remaining within practical time bounds (RQ_4), as detailed next.

4.3.4 RQ_4. Comparative Performance

PROMISE is compared against the 16 baselines introduced in Sec. 4.2.5: nine RAOM4CC heuristics [14], six EdgeWiseCR strategies [12], and MS-GD-P [9]. All 17 techniques were executed over the full benchmark of 9,600 scenarios, yielding 163,200 data points.

Because several baseline variants behave nearly identically, we report results at two levels of granularity. Tab. 4.3 provides a grouped summary that aggregates the 16 baselines into five families (RAOM4CC `one_layer_*`, RAOM4CC advanced, EdgeWiseCR greedy, EdgeWiseCR MILP, and MS-GD-P) alongside PROMISE. A detailed discussion of the results in Tab. 4.3 can be found in the main paper. The subsequent subsections then disaggregate the results per individual variant (Tab. 4.4–4.6) and application type, which is the level at which the most informative differences emerge.

Feasibility

Because the baselines do not model all the constraints captured by PROMISE, a solution deemed feasible by a baseline may still violate constraints enforced by PROMISE, most notably the provider exclusion relations between OPTUS and TELSTRA. Figure 4.3 re-evaluates every baseline solution against that additional constraint set, which considers interoperability agreements among providers and the demand for specific node types in the selected configuration. Under this set of constraints the picture changes materially: the EdgeWiseCR MILP variants (`edgewise`, `edgewise_cr`, `edgewise_num`) collapse to 8–64% across applications, because

Table 4.3: Summary of RQ_4 results. Median values and IQ ranges are presented. Highlighted rows guarantee optimality.

Technique	Time (s)	Nodes	Cost ($)	Feasibility (%)		
				Prov.	+Crit.	+All
RAOM4CC `one_layer_*` [14]	0.0001	1	16.0–1626.2	21–97	0–14.6	0
RAOM4CC advanced [14]	0.0005	1	16.0–1658.0	100	0	0
EdgeWiseCR greedy [12]	0.0006	1	15.2–1551.6	100	0	0
EdgeWiseCR MILP [12]	0.0025	6	15.2–1481.6	26–41	3.1–9.4	0–2.1
MS-GD-P genetic [9]	0.01	16	61.2–13750	11.8	5.9	2.1
PROMISE (S/M scale)	0.1	3	37.8–751.27	100	100	100
PROMISE (L scale)	0.34	3–4	52.7–1607.5	100	100	100

their multi-node deployments frequently co-locate OPTUS and TELSTRA devices that exclude each other; MS-GD-P suffers the steepest collapse, dropping to 5.8–17.0% across applications (11.4% overall), because its priority-driven genetic search maximises coverage without checking provider compatibility; the RAOM4CC multi-layer heuristics that select across more than one node (`round_robin`, `one_layer_edge`) drop to 88–100%, reflecting sporadic co-deployments of mutually exclusive providers; and the variants that select a single node cannot violate the exclusion by construction, so their rate equals the original feasibility rate. PROMISE remains at 100% across all applications.

Tab. 4.4 reports the recomputed feasibility rate together with the median node count selected by each technique. The node count explains the structural pattern: single-node heuristics are immunized against provider exclusions by construction, whereas every multi-node heuristic incurs a non-trivial violation rate. The three overall-rate columns progressively activate additional constraints that only PROMISE enforces: the device-type filter, checked in two increasingly strict forms –coverage of the application's *critical* sensor type (CCTV/VR→CAMERA, LiDAR/Robot→SENSOR) and coverage of *all* device types required by the application.

Table 4.4: Feasibility rate (%) and node-selection profile, by technique. Per-application columns report the rate under demand satisfaction plus provider exclusion (the six constraints shared by PROMISE and the baselines). The three overall-rate columns progressively activate additional constraints that only PROMISE enforces: the critical device-type filter and the full device-type coverage. The median is taken over feasible solutions under each technique's own model; the range reports the minimum and maximum node count across those solutions.

Technique	Per-application (%)				Overall (%)			Med.	Range
	CCTV	LiDAR	Robot	VR	Prov.	+Crit.	+All	Nodes	
PROMISE	100.0	100.0	100.0	100.0	100.0	100.0	100.0	3	2–6
RAOM4CC `one_layer_mist`	0.0	0.0	83.3	0.0	20.8	14.6	0.0	1	1–2
RAOM4CC `one_layer_edge`	96.9	87.5	100.0	100.0	96.9	0.0	0.0	1	1–3
RAOM4CC `one_layer_cloud`	72.9	91.7	0.0	100.0	72.9	0.0	0.0	1	1–2
RAOM4CC `round_robin`	97.9	91.7	100.0	100.0	97.9	0.0	0.0	1	1–3
RAOM4CC `best_fit`	100.0	100.0	100.0	100.0	100.0	0.0	0.0	1	1–2
RAOM4CC `best_fit_delay`	100.0	100.0	100.0	100.0	100.0	0.0	0.0	1	1–2
RAOM4CC `best_fit_delay_energy`	100.0	100.0	100.0	100.0	100.0	0.0	0.0	1	1–2
RAOM4CC `delay_heuristics`	100.0	100.0	100.0	100.0	100.0	0.0	0.0	1	1–2
RAOM4CC `delay_energy_heuristics`	100.0	100.0	100.0	100.0	100.0	0.0	0.0	1	1–2
EdgeWiseCR `prolog`	100.0	100.0	100.0	100.0	100.0	0.0	0.0	1	1–2
EdgeWiseCR `prolog_cr`	100.0	100.0	100.0	100.0	100.0	0.0	0.0	1	1–2
EdgeWiseCR `prolog_num`	100.0	100.0	100.0	100.0	100.0	0.0	0.0	1	1–2
EdgeWiseCR `edgewise`	33.3	8.3	29.2	33.3	26.0	3.1	2.1	6	1–10
EdgeWiseCR `edgewise_cr`	33.3	8.3	29.2	33.3	26.0	3.1	2.1	6	1–10
EdgeWiseCR `edgewise_num`	33.3	29.2	66.7	33.3	40.6	9.4	0.0	6	1–10
MS-GD-P	9	8.7	13.1	16.5	11.8	5.9	2.1	16	3–80

The progression across the three overall-rate columns tells a single, layered story. Under the *provider exclusion* constraint alone, the strategies that select more than two nodes lose feasibility

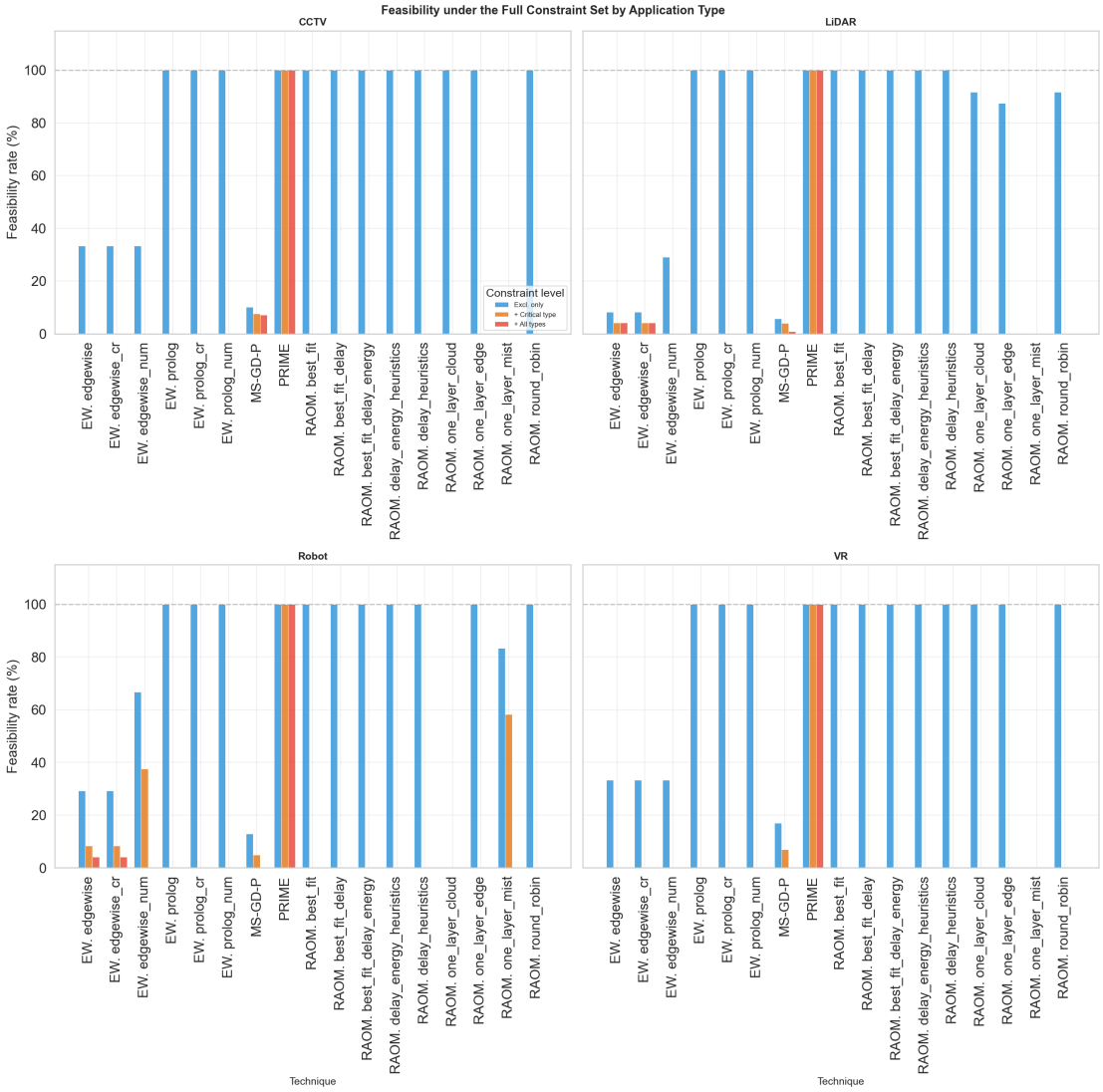


Figure 4.3: Feasibility rate under the full constraint set (demand satisfaction plus provider exclusion), recomputed for every baseline solution against the same definition PROMISE solves. Facets correspond to the four application types; bars are grouped by technique.

(EdgeWiseCR MILP drops to 26–41%, MS-GD-P to 11.4%), while single-node heuristics remain at 100% only because the exclusion is vacuous for a single provider. Activating the *critical device-type* requirement exposes the single-node artefact immediately: every single-node strategy except RAOM4CC `one_layer_mist` (which finds mist-tier SENSOR devices for Robot in 58.3% of those scenarios) collapses from 100% to 0%, EdgeWiseCR MILP falls further to 3–9%, and MS-GD-P holds at 10.1% because its multi-node deployments occasionally include a provider whose catalogue offers the critical sensor type. Activating the *full device-type coverage* requirement drives every baseline to near-zero feasibility (0–2.1%, see the *+All* column of Tab. 4.3), with only the occasional EdgeWiseCR MILP `edgewise` variant on LiDAR/Robot ($< 5\%$) and MS-GD-P (2.1%) remaining above zero. PROMISE retains 100% across all three columns because its solver co-locates exactly the set of add-ons needed to cover every required device type without violating provider exclusions, selecting 2–6 nodes (median 3) per solution.

Two additional patterns emerge in the single-layer RAOM4CC variants, visible in the per-application columns of Tab. 4.4. First, `one_layer_mist` fails completely for CCTV, LiDAR, and VR because mist-tier devices lack the GPU and storage capacity that these workloads demand; Robot applications, which require heterogeneous sensors and actuators rather than heavy compute, find some mist devices suitable –hence the partial 83.3% feasibility. Second, `one_layer_cloud` exhibits the complementary failure: it achieves 100% for CCTV and VR (which

benefit from cloud-tier GPU capacity) but collapses to 0% for Robot, where the required device types (sensors, actuators) reside at the edge and mist tiers rather than in cloud data centres. LiDAR shows a partial 91.7% because some LiDAR scenarios require device types absent from the cloud tier. The `one_layer_edge` variant succeeds universally because edge nodes occupy a middle ground in both capacity and device diversity: when a single-tier heuristic is necessary, edge-only placement is the only safe default.

Execution Time

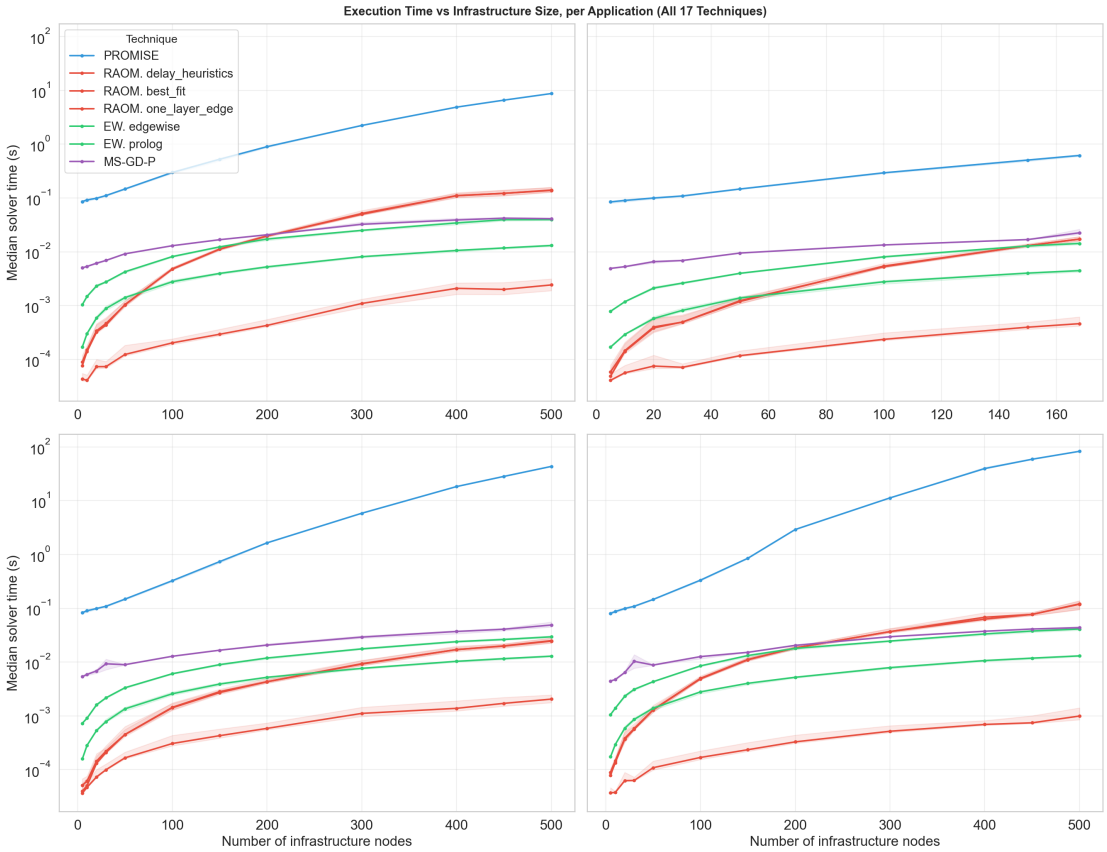


Figure 4.4: Execution time versus infrastructure size per application type. Each point is one scenario instance; techniques are distinguished by colour. Baseline curves remain flat across infrastructure sizes, whereas PROMISE steepens dramatically for VR and Robot beyond 200 nodes.

PROMISE's solver time grows at vastly different rates across the four application types. At small scale (5–30 infrastructure nodes), all four workloads complete in under 0.16 s (median 0.098 s, Tab. 4.5), and the differences between applications are negligible. Beyond approximately 200 nodes, the computational cost diverges sharply, as shown in Figure 4.4: VR and Robot steepen dramatically while LiDAR remains nearly flat. At large scale, PROMISE's median time reaches 0.34 s (Tab. 4.3) with a global maximum of 83.8 s (Tab. 4.5), an increase of more than two orders of magnitude over the small-scale baseline. The divergence reflects the combinatorial complexity of each workload's constraint structure: VR scenarios involve the highest user counts, the most diverse feature requirements, and the tightest latency constraints, producing a constraint network that the CP solver must explore extensively.

Unlike PROMISE, the 16 baselines exhibit near-constant execution times across application types, because their heuristic and MILP formulations process a fixed constraint set whose size does not vary with workload complexity. Tab. 4.5 reports the median execution time together with the per-technique interval (min–max) aggregated over all four applications, by scenario scale. At small scale, every technique's interval is tight and the medians cluster within two orders of magnitude (PROMISE 0.098 s vs. RAOM4CC `one_layer_*` 7.8e-5 s). From medium scale onwards PROMISE's upper bound explodes on VR and Robot: at large scale its global

interval reaches [0.088–83.8] s, whereas the widest baseline interval (EdgeWiseCR MILP at L) stays below 0.07 s –three orders of magnitude tighter than PROMISE's upper tail. The per-variant table confirms that the ordering criterion within each family (round-robin vs. best-fit vs. delay-aware for RAOM4CC; cost vs. cost+reliability for EdgeWiseCR) has negligible impact on execution time, which justifies the grouped presentation of Tab. 4.3.

Table 4.5: Global execution time (seconds) aggregated over all four applications, by technique and scenario scale (feasible solutions only). Cells report `median [min--max]`.

Technique	**S (all apps)**	**M (all apps)**	**L (all apps)**
PROMISE	0.098 [0.072–1.69]	0.131 [0.087–2.98]	0.34 [0.088–83.8]
RAOM4CC `one_layer_mist`	7.8e-5 [3.3e-5–6.0e-4]	1.4e-4 [5.8e-5–4.3e-3]	4.3e-4 [5.9e-5–2.9e-2]
RAOM4CC `one_layer_edge`	7.9e-5 [3.4e-5–5.8e-4]	1.4e-4 [5.9e-5–4.1e-3]	4.4e-4 [6.0e-5–2.8e-2]
RAOM4CC `one_layer_cloud`	7.7e-5 [3.4e-5–5.7e-4]	1.3e-4 [5.7e-5–4.0e-3]	4.2e-4 [5.8e-5–2.8e-2]
RAOM4CC `round_robin`	3.0e-4 [3.7e-5–3.2e-3]	5.8e-4 [1.2e-4–5.4e-2]	7.6e-3 [1.2e-4–3.2e-1]
RAOM4CC `best_fit`	3.1e-4 [3.8e-5–3.1e-3]	5.9e-4 [1.3e-4–5.3e-2]	7.4e-3 [1.3e-4–3.1e-1]
RAOM4CC `best_fit_delay`	3.1e-4 [3.9e-5–3.1e-3]	6.0e-4 [1.3e-4–5.3e-2]	7.5e-3 [1.3e-4–3.1e-1]
RAOM4CC `best_fit_delay_energy`	3.2e-4 [4.0e-5–3.2e-3]	6.1e-4 [1.4e-4–5.4e-2]	7.6e-3 [1.4e-4–3.2e-1]
RAOM4CC `delay_heuristics`	3.0e-4 [3.7e-5–3.1e-3]	5.8e-4 [1.2e-4–5.3e-2]	7.4e-3 [1.2e-4–3.1e-1]
RAOM4CC `delay_energy_heuristics`	3.1e-4 [3.8e-5–3.2e-3]	5.9e-4 [1.3e-4–5.4e-2]	7.6e-3 [1.3e-4–3.2e-1]
EdgeWiseCR `prolog`	5.3e-4 [1.3e-4–1.6e-3]	1.1e-3 [4.5e-4–6.2e-3]	2.5e-3 [4.8e-4–2.0e-2]
EdgeWiseCR `prolog_cr`	5.4e-4 [1.4e-4–1.6e-3]	1.1e-3 [4.6e-4–6.3e-3]	2.6e-3 [4.9e-4–2.0e-2]
EdgeWiseCR `prolog_num`	5.5e-4 [1.5e-4–1.7e-3]	1.2e-3 [4.7e-4–6.4e-3]	2.7e-3 [5.0e-4–2.1e-2]
EdgeWiseCR `edgewise`	2.1e-3 [7.0e-4–2.3e-2]	3.1e-3 [1.5e-3–2.7e-2]	8.8e-3 [1.3e-3–6.4e-2]
EdgeWiseCR `edgewise_cr`	2.2e-3 [7.1e-4–2.4e-2]	3.2e-3 [1.6e-3–2.8e-2]	9.0e-3 [1.4e-3–6.5e-2]
EdgeWiseCR `edgewise_num`	2.3e-3 [7.2e-4–2.5e-2]	3.3e-3 [1.7e-3–2.9e-2]	9.2e-3 [1.5e-3–6.6e-2]
MS-GD-P	3.8e-3 [3.0e-3–4.7e-3]	5.3e-3 [3.9e-3–2.9e-2]	1.1e-2 [4.0e-3–4.5e-2]

Node Selection

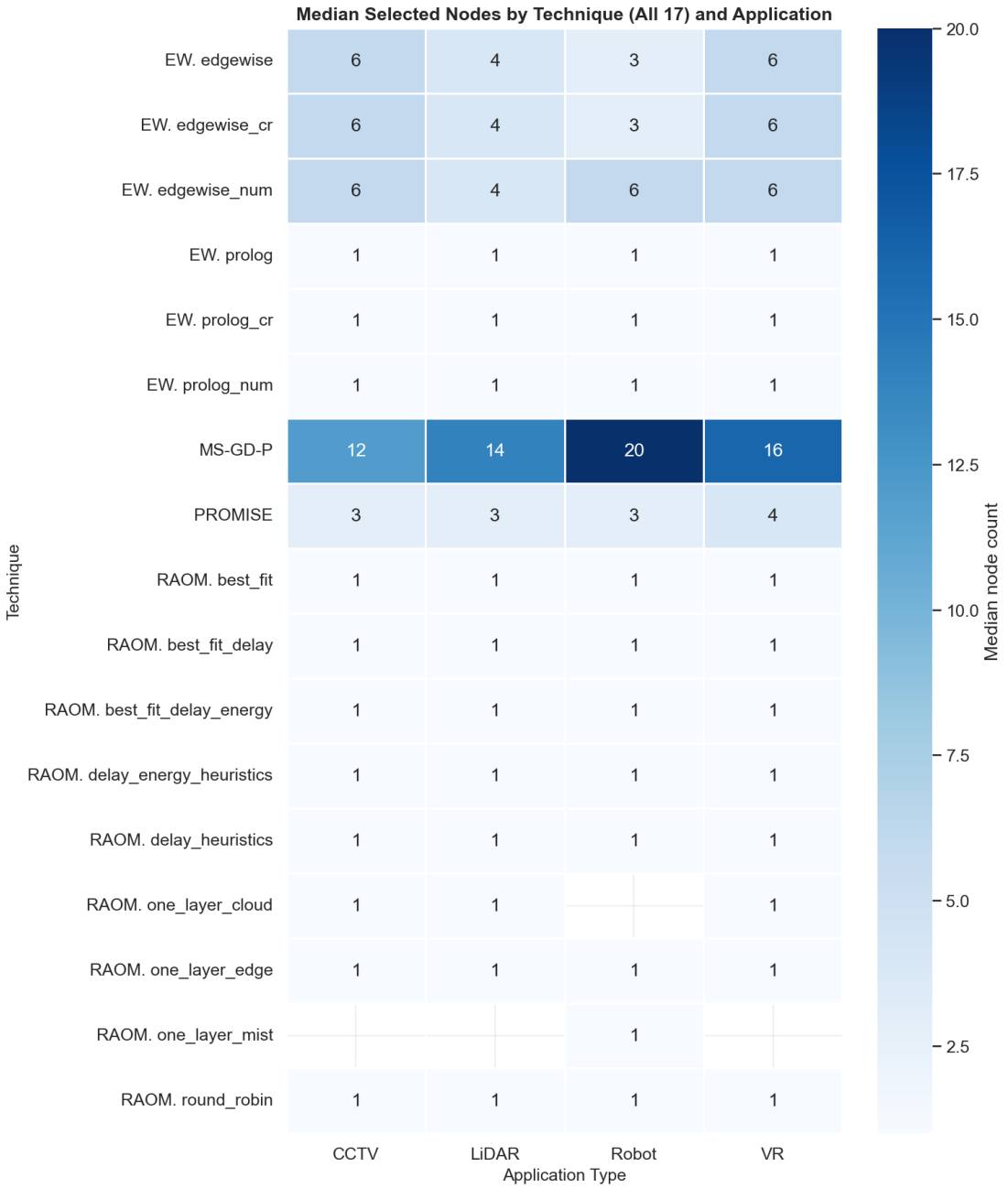


Figure 4.5: Median number of nodes selected by each technique, by application type (feasible solutions only).

The number of nodes that each technique selects varies by application type, particularly for PROMISE and EdgeWiseCR MILP, as shown in Figure 4.5. Overall, all RAOM4CC and EdgeWiseCR greedy variants select one to two nodes regardless of workload (Tab. 4.3), because their greedy and single-layer strategies co-locate resources on the first feasible device. EdgeWiseCR MILP distributes demand across more nodes (3–10), with higher counts on CCTV and VR than on LiDAR and Robot, because its MILP formulation benefits from aggregation when the workload's resource profile is heterogeneous. MS-GD-P selects the most nodes (median 20, range 3–100; Figure 4.5), as its genetic algorithm maximises coverage by spreading demand across many devices. PROMISE selects 3–4 nodes, reflecting the constraint solver's balancing of cost minimization against feature coverage and provider exclusion constraints. The practical trade-off is between management overhead (fewer nodes are simpler to operate) and resource adequacy (more nodes provide better coverage and lower per-node load); PROMISE's selection represents a middle ground, sufficient to satisfy feature and exclusion constraints but compact enough to

avoid excessive management complexity.

Solution Cost

The raw cost figures reported by each technique are not directly comparable, but they warrant analysis because they reveal the structural mechanism by which the baselines achieve their apparent advantage. Tab. 4.6 reports the global solution-cost range (min–max) aggregated over all four applications, by technique and scenario scale. Two regularities emerge. First, the cost range of most techniques widens from S to L, because larger user populations raise demand and admit richer (and pricier) infrastructures; PROMISE's global maximum rises from \$794.18 (S) to \$1607.49 (L), a 2× increase, and the baselines exhibit the same trend (e.g., EdgeWiseCR greedy from \$15.24–\$468.01 to \$16.21–\$1551.64). MS-GD-P is the exception: its cost range is constant at \$135.31–\$165.31 across scales, because its node selections and pricing do not vary with scale. Second, PROMISE's lower bound is consistently above the baselines' lower bound across all scales (e.g., \$37.83 vs. \$15.15 at S scale), and its upper bound typically exceeds the baselines' upper bound –reflecting the deployability premium its constraint coverage commands rather than an efficiency deficit.

Table 4.6: Global solution-cost range (min–max, \$) aggregated over all four applications, by technique and scenario scale (feasible solutions only).

Technique	S (all apps)	M (all apps)	L (all apps)
PROMISE	37.83–794.18	52.70–751.27	52.70–1607.49
RAOM4CC `one_layer_mist`	16.02–468.01	18.12–615.52	18.90–1626.24
RAOM4CC `one_layer_edge`	16.02–468.01	18.12–615.52	18.90–1626.24
RAOM4CC `one_layer_cloud`	16.02–468.01	18.12–615.52	18.90–1626.24
RAOM4CC `round_robin`	16.02–412.95	18.12–579.65	18.90–1658.00
RAOM4CC `best_fit`	16.02–412.95	18.12–579.65	18.90–1658.00
RAOM4CC `best_fit_delay`	16.02–412.95	18.12–579.65	18.90–1658.00
RAOM4CC `best_fit_delay_energy`	16.02–412.95	18.12–579.65	18.90–1658.00
RAOM4CC `delay_heuristics`	16.02–412.95	18.12–579.65	18.90–1658.00
RAOM4CC `delay_energy_heuristics`	16.02–412.95	18.12–579.65	18.90–1658.00
EdgeWiseCR `prolog`	15.24–468.01	15.80–615.52	16.21–1551.64
EdgeWiseCR `prolog_cr`	15.24–468.01	15.80–615.52	16.21–1551.64
EdgeWiseCR `prolog_num`	15.24–468.01	15.80–615.52	16.21–1551.64
EdgeWiseCR `edgewise`	15.15–398.01	15.50–545.52	15.75–1481.64
EdgeWiseCR `edgewise_cr`	15.15–398.01	15.50–545.52	15.75–1481.64
EdgeWiseCR `edgewise_num`	15.15–398.01	15.50–545.52	15.75–1481.64
MS-GD-P	61–1874	348.03–5351	2441–13750

Why the costs are not comparable. Three structural factors inflate PROMISE's reported cost relative to the baselines, and none of them reflects inefficiency on PROMISE's part. *(i) Constraint coverage:* the six constraints that only PROMISE enforces eliminate the cheapest solutions the baselines can select; a baseline that places two incompatible providers on the same node reports a low cost because it ignores the exclusion, whereas PROMISE must either select a different provider or add a second node. *(ii) Node count:* the single-node baselines report only one device's price, while PROMISE reports the sum of 2–6 selected nodes' prices; the comparison therefore confounds constraint satisfaction with infrastructure quantity. *(iii) Pricing formalism:* PROMISE uses symbolic price expressions that account for subscription minimums, usage limits, and renewable/non-renewable resource tiers, whereas the baselines use a simplified per-unit cost model that ignores subscription and tier awareness.

Deployability trumps nominal cost. Despite reporting higher nominal costs, PROMISE produces structurally superior solutions to the single-node baselines on every application type. The superiority is not a matter of cost efficiency but of solution adequacy: a single-node solution cannot, in general, satisfy provider exclusions that span two providers, the full feature type set, or subscription minimum-quantity constraints. Tab. 4.3 and Tab. 4.6 make this structural limitation visible: the single-node baselines report low costs (e.g., \$15–\$16 at S scale) precisely because they select one to two nodes and ignore the six additional constraints, whereas PROMISE

selects 3–4 nodes and reports the resulting $37.83–$1607.49 cost. A deployment that uses a single-node baseline solution would require manual post-hoc validation against every constraint the baseline skipped, and would need to add nodes (and cost) to fix violations; PROMISE's reported cost already includes this constraint satisfaction. EdgeWiseCR MILP occupies an intermediate position: it selects 3–10 nodes and reports costs close to the single-node baselines, because its MILP minimizes cost over the simplified constraint set; MS-GD-P selects even more nodes (median 20) at a higher cost ($135.31–$165.31), but its 11.4% feasibility under provider exclusion shows that coverage maximisation does not enforce constraint completeness. In both cases, the cost advantage disappears when the missing constraints are enforced. We therefore conclude that the cost figures across techniques are not comparable: the baselines report lower costs because they solve a relaxed problem, whereas PROMISE reports the cost of a deployable solution.

4.4 Threats to Validity

Despite the positive results obtained in our evaluation, several threats to validity must be acknowledged.

Internal Validity. The main threat to internal validity relates to the synthetic generation of topologies and demand workloads. While these values were designed to be plausible and were grounded in established capacity planning abstractions, they may not perfectly capture the complex, non-linear resource dynamics of production-grade scenarios.

External Validity. The generalizability of our findings is limited by the use of a single geographically concentrated dataset (EUA) focused on the Melbourne metropolitan area. Although we evaluated 9,600 test cases across 96 distinct scenario types, different infrastructure distributions or provider ecosystems with more complex interoperability rules might yield different performance results. Furthermore, our evaluation considered a fixed set of three providers, which may not represent the full diversity of the global computing continuum market, although it is plausible for many scenarios.

Construct Validity. A potential threat to construct validity lies in the use of geographic distance as a proxy for latency when defining request constraints. While this is a common simplification in edge computing research [11], it ignores other network factors such as congestion and routing hops that affect actual service-level objectives.

Conclusion Validity. To mitigate threats to conclusion validity, each scenario type was instantiated 100 times and results are reported as median execution times with their corresponding confidence intervals, reducing the impact of incidental topologies and transient system load.

Chapter 5

Conclusions and Future Work

This paper set out to answer whether a pricing-based formulation is expressive enough to model multi-provider resource allocation scenarios ($\mathbf{RQ}_1$); whether it can compute cost-optimal deployments ($\mathbf{RQ}_2$); how it scales as demand and offer increase ($\mathbf{RQ}_3$); and how it compares against state-of-the-art resource allocation techniques in terms of feasibility, performance, and solution quality ($\mathbf{RQ}_4$).

For $\mathbf{RQ}_1$, we successfully represented 9,600 RA problems –spanning four application types, three deployment scales, and provider interoperability agreements– as iPricings. For $\mathbf{RQ}_2$, PROMISE consistently identified cost-optimal configurations when solutions existed and correctly reported infeasibility otherwise. For $\mathbf{RQ}_3$, execution time remained largely insensitive to demand and increased predictably with the number of candidate nodes, with topologies of up to 200 nodes solved in under one second in most cases. For $\mathbf{RQ}_4$, PROMISE outperformed 17 state-of-the-art baselines in feasibility and node selection.

Several research directions emerge from this work. Extending PROMISE with monitoring and dynamic re-optimization mechanisms would enable adaptive resource management, while validating the approach on production infrastructures and real workloads would strengthen its external validity. Beyond resource allocation, the proposed pricing abstraction could be further generalized to other configuration problems, such as QoS-aware service composition. Finally, hybrid solving strategies, decomposition techniques, and dedicated heuristics may further improve performance on scenarios comprising thousands of candidate nodes.

Bibliography

[1] Anonymous: Pricing-Driven Resource Allocation in the Cloud Continuum – Laboratory Package (2026), https://doi.org/10.5281/zenodo.19063676

[2] Barroso, L.A., Clidaras, J., Hölzle, U.: The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines. Synthesis Lectures on Computer Architecture, Morgan & Claypool Publishers (2013). https://doi.org/10.2200/S00516ED2V01Y201306CAC024

[3] Crankshaw, D., Wang, X., Zhou, G., Franklin, M.J., Gonzalez, J.E., Stoica, I.: Clipper: A low-latency online prediction serving system. In: Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI). USENIX Association (2017)

[4] García-Fernández, A., Parejo, J.A., Ruiz-Cortés, A.: Pricing4SaaS: Towards a pricing model to drive the operation of SaaS. In: International Conference on Advanced Information Systems Engineering. pp. 47–54 (2024)

[5] García-Fernández, A., Parejo, J.A., Trinidad, P., Ruiz-Cortés, A.: Automated Analysis of Pricings in SaaS-Based Information Systems. In: International Conference on Advanced Information Systems Engineering. pp. 223–239 (2025)

[6] García-Fernández, A., Parejo, J.A., Cavero, F.J., Ruiz-Cortés, A.: Trends in industry support for pricing-driven devops in saas. IEEE Transactions on Services Computing **19**(1), 726–737 (2026)

[7] García-Fernández, A., Sedlak, B., Parejo, J.A., Frangoudis, P., Ruiz-Cortés, A., Dustdar, S.: Towards a unified view of configuration problems in the computing continuum. In: IEEE International Conference on Web Services (ICWS) (2026), in press

[8] Jin, H., Wang, H., Luo, J.: MS-GD-P: priority-based service deployment for cloud-edge-end scenarios. The Journal of Supercomputing **80**(18), 25713–25735 (Dec 2024). https://doi.org/10.1007/s11227-024-06423-z

[9] Jin, H., Wang, H., Luo, J.: Ms-gd-p: priority-based service deployment for cloud-edge-end scenarios: H. jin et al. The Journal of Supercomputing **80**(18), 25713–25735 (2024)

[10] Lai, P.: PhuLai/eua-dataset (Dec 2025), https://github.com/PhuLai/eua-dataset, original-date: 2018-06-05T08:32:30Z

[11] Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.B.: A survey on mobile edge computing: The communication perspective. IEEE communications surveys & tutorials **19**(4), 2322–2358 (2017)

[12] Massa, J., Forti, S., Dazzi, P., Brogi, A.: Combining declarative and linear programming for application management in the cloud-edge continuum. Future Generation Computer

Systems p. 108224 (2025)

[13] Menascé, D.A., Almeida, V.A.F., Dowdy, L.W.: Capacity Planning for Web Services: Metrics, Models, and Methods. Prentice Hall (2002)

[14] Mihaiu, M., Mocanu, B.C., Negru, C., Petrescu-Niță, A., Pop, F.: Resource allocation optimization model for computing continuum. Mathematics **13**(3), 431 (2025)

[15] Molino-Peña, E., García, J.M., Ruiz-Cortés, A.: Integrating terms of service and service level agreements for automating cloud service management. In: International Conference on Service-Oriented Computing. pp. 19–34. Springer (2025)

[16] Shen, Z., Subbiah, S., Gu, X., Wilkes, J.: Cloudscale: Elastic resource scaling for multi-tenant cloud systems. IEEE Transactions on Parallel and Distributed Systems **23**(2), 259–266 (2011). https://doi.org/10.1109/TPDS.2011.229

[17] Soumplis, P., Kokkinos, P., Kretsis, A., Nicopolitidis, P., Papadimitriou, G., Varvarigos, E.: Resource allocation challenges in the cloud and edge continuum. In: Advances in Computing, Informatics, Networking and Cybersecurity: A Book Honoring Professor Mohammad S. Obaidat's Significant Scientific Contributions, pp. 443–464. Springer (2022)

[18] Vergara, J., Botero, J., Fletscher, L.: A comprehensive survey on resource allocation strategies in fog/cloud environments. Sensors **23**(9), 4413 (2023)

[19] Zeng, L., Benatallah, B., Ngu, A.H., Dumas, M., Kalagnanam, J., Chang, H.: Qos-aware middleware for web services composition. IEEE Transactions on software engineering **30**(5), 311–327 (2004)